

Conceptual Architecture Report of Gemini CLI

Feb 13, 2026

Authors:

Mehrin Hossain (23nt1@queensu.ca)
Lauren Sutherland (21las46@queensu.ca)
Jaineel Modi (22rsw2@queensu.ca)
Sophia Kim (18yjk@queensu.ca)
Minh Quang Tran (22rfx@queensu.ca)
Michael Okene (22gvl2@queensu.ca)

1 Introduction.....	2
2 Gemini CLI Architectural Overview.....	2
2.1 Overall Architectural Principles.....	2
2.2 Analysis of Major Subsystems.....	4
2.2.1 CLI (Command-Line Interface) Subsystem.....	4
2.2.2 Session Manager.....	5
2.2.3 Agent Core/Orchestration Subsystem.....	5
2.2.4 Gemini API Client Subsystem.....	6
2.2.5 Local Tooling Subsystem.....	7
2.2.6 External Tooling Subsystem.....	7
2.3 Interplay of Components.....	8
3 External Interfaces.....	8
3.1 Division of Responsibilities Among Developers.....	9
4 Use Cases.....	10
4.1 Use Case 1: Refactor a File.....	10
4.2 Use Case 2: Run Tests Safely and Summarize Failures (Approval + Sandboxed Shell)...	11
5 Conclusion.....	12
6 Lessons Learned.....	12
7 Data Dictionary.....	13
8 Naming Conventions.....	14
9 References.....	14
10 AI Report.....	15

Abstract

Gemini CLI is an AI-based command-line interface assistant intended for developer assistance in various software code engineering activities like code navigation, code refactoring, code testing, and automation procedures. The tool combines a large language model and a CLI interface to provide a CLI-based platform where users can interface with other complex tools using natural language input methods. From the perspective of computer programming, the Gemini CLI has various high-level components, which include the command-line interface, an agent core where the orchestration of the intended task is conducted, a tooling component where system and development tool commands are run, an LLM integration component, a context and memory component, and an interface component. From an architectural point of view, Gemini CLI is designed using a client server architectural style, with a layered organization applied within its client and server components, and agent-based orchestration used internally to coordinate reasoning and tool execution. Therefore, it is clear that the Gemini system has been developed in a modular fashion, which allows for easy extension and evolution of the system over time. The major strengths of Gemini CLI revolve around its modularity, flexibility, and capacity to improve the productivity of developers by providing complex interactions based on conversational interfaces. Nevertheless, the major weaknesses of the developed system include its use of large language models, the possibility of having increased latency, as well as limitations when the system is regarded as offline.

1 Introduction

The objective of this report is to establish the conceptual architecture of Gemini CLI, aiming at assisting new developers and stakeholders in gaining a better understanding of the architecture and abstractions of the main components. Whereas other reports may concentrate on implementation issues and source code, as a high-level architecture-focused document, this one puts more emphasis in this area. The system at hand is an AI-assisted tool for developers built as a command-line interface-based tool. Gemini CLI harnesses the benefits of a combination of CLI-based approaches to developer tooling and an agent-based orchestration system infused with the power of large language model AI reasoning. Therefore, the system under discussion is a confluence of developer tools, agent systems, and AI-based automation systems. This report is organized as follows; First, an architectural overview describes the overall structure of Gemini CLI and its guiding design principles. Next, the major subsystems are examined and their roles within the architecture are outlined. The report then presents key use cases illustrated through sequence diagrams to demonstrate control and data flow across the system. Finally, conclusions and lessons learned summarize architectural insights and challenges encountered during the analysis, followed by an appendix documenting collaboration with an AI teammate.

2 Gemini CLI Architectural Overview

2.1 Overall Architectural Principles

GeminiCLI is structured as a client-server system with a layered organization applied across both the client and server components. A lightweight command line client interacts with centralized backend services to perform large language model inference and controlled tool execution. This separation reflects the client-server architectural style described by Garlan and Shaw, in which clients request services from centralized servers through well defined communication channels. From a requirements perspective, this architectural decision supports key non-functional requirements such as scalability, centralized evolution, and maintainability.

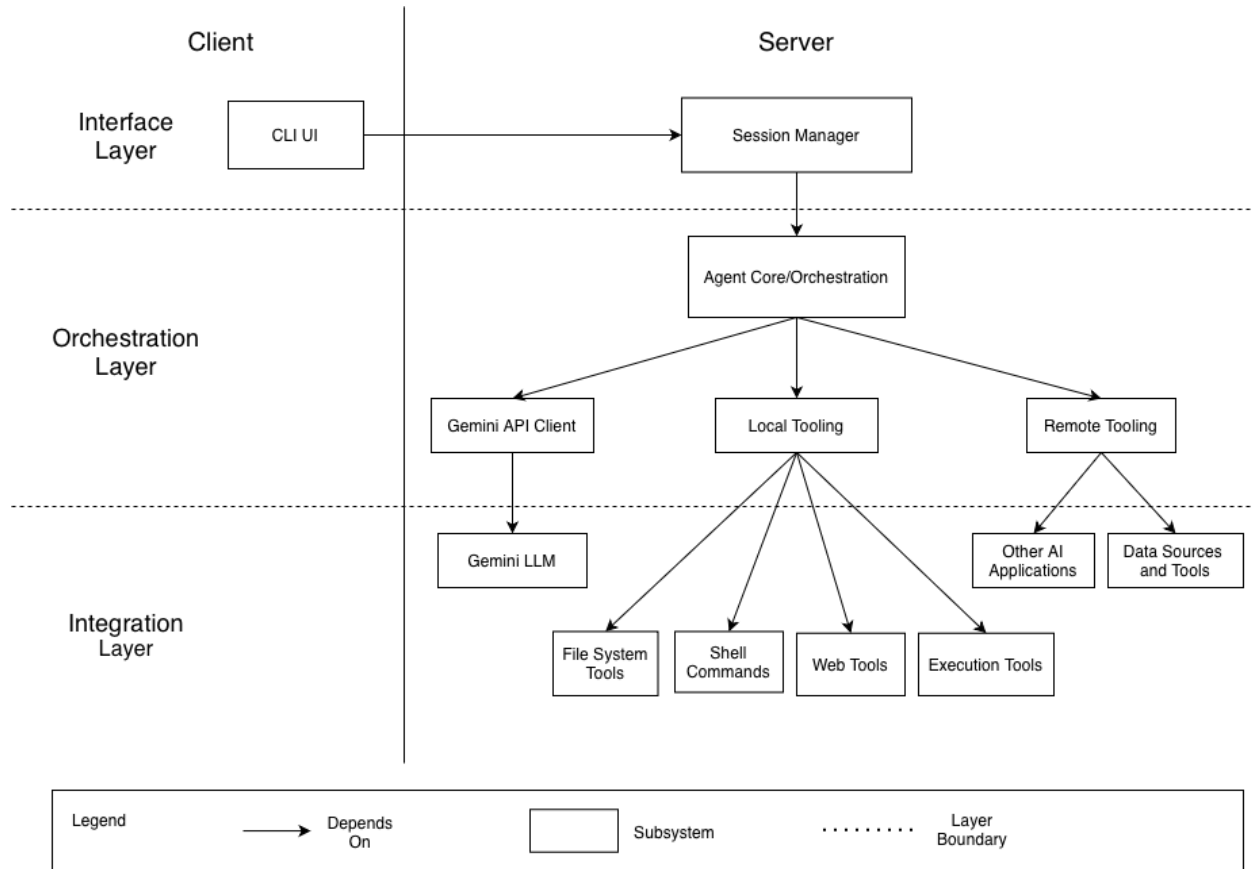


Figure 1: High-level conceptual architecture diagram of Gemini CLI

As shown in Figure 1, the architecture is organized into three primary layers: the Interface Layer, the Orchestration Layer, and the Integration Layer. The Interface Layer, located on the client side, consists of the CLI UI and the Session Manager. This layer isolates presentation concerns from backend reasoning and execution logic, improving usability and testability. On the client side, the CLI UI captures user input and forwards requests to the Session Manager on the server side. The Session Manager resides in the server's Interface Layer and is responsible for maintaining session state, preserving conversational context, and preparing requests before passing control to the orchestration components. Beneath the Interface Layer lies the Orchestration Layer. This layer includes the Agent Core / Orchestration subsystem, along with the Gemini API Client, Local Tooling, and Remote Tooling modules. The Agent Core overwatches global control flow by interpreting user intent and determining which subsystems must be invoked. It represents language model inference to the Gemini API Client, routes system level actions to the Local Tooling subsystem, and directs external service interactions to the Remote Tooling subsystem. These components remain within the Orchestration Layer to ensure centralized coordination and prevent reasoning components from directly interacting with execution targets. The Integration Layer contains the concrete execution targets that perform environmental and external actions. The Gemini API Client communicates downward to the Gemini LLM. The Local Tooling subsystem interacts with File System Tools, Shell Commands,

Web Tools, and Execution Tools within the local environment. The Remote Tooling subsystem connects to Other AI Applications and external Data Sources and Tools. By isolating these execution mechanisms within the Integration Layer, the architecture cleanly separates coordination logic from side effecting operations, reinforcing modularity and reducing unintended coupling.

Global control flow proceeds from the CLI UI to the Agent Core, which then delegates work to either the Gemini API Client or the appropriate tooling subsystem. Responses return through the same layered path and are streamed back to the CLI UI for incremental display. Concurrency is handled primarily through asynchronous message passing and streaming rather than heavy multithreading, allowing the system to remain responsive during network-bound interactions. Performance critical operations, particularly language model inference and tool execution, occur within the Orchestration and Integration Layer. Because these operations may introduce network latency, the architecture minimizes blocking behavior in the Interface Layer and supports streamed responses to maintain responsiveness. Evolvability is supported through clear separation between layers. New tools can be introduced in the Integration Layer without altering orchestration logic, and backend language models can be upgraded or replaced without modifying the Interface Layer. This layered separation reduces architectural ripple effects and supports long-term maintainability. Overall, GeminiCLI combines a client server structure with a layered organization and agent-based orchestration. This design prioritizes modularity, controlled execution, responsiveness, and adaptability while explicitly balancing trade-offs among scalability, performance, and modifiability.

2.2 Analysis of Major Subsystems

2.2.1 CLI (Command-Line Interface) Subsystem

The CLI subsystem is the user-facing interaction manager that handles all direct communication with the user via the command line. It handles user input and output, as well as managing all components of the user's experience. This includes things like capturing user input, keeping track of session history, rendering the display, providing theme/UI customization options, and setting up personalized CLI configuration methods for behavior. This subsystem conceptually constitutes the interface layer in a way that sets up a clear separation of concerns from the frontend and the backend by focusing on the development and maintenance of the command line interface. This subsystem allows for an interactive terminal interface as opposed to a static terminal seen with other command line tools, which lends itself to Gemini CLI's ethos of natural and customized AI assistance that goes beyond text generation.

2.2.2 Session Manager

The Session Manager is a server-side component that helps make up the interface layer through its proximity to the user and its capacity as a bridge between the client-facing UI and the core logic. It handles session authentication, by making sure that the client is authorized to make requests and stores tool permissions from the user. In addition, the session manager is used to save the user's conversation history, including the user's prompts and the model's responses, tool executions (including input and output), and keeps track of token usage. It is also used to keep track of the project scope by tracking the environment settings and updating the context whenever needed. In doing so, it'll also keep track of currently running tools to make it easier to resume the conversation, save its state, and prevent overlapping changes.

2.2.3 Agent Core/Orchestration Subsystem

The Agent Core subsystem helps to make up the orchestration layer that is concerned with processing, reasoning, and coordination. Once the orchestration layer receives requests sent from the interface layer, the Agent Core/Orchestration is responsible for the decision-making of actions that are necessitated by the prompt or context. It also handles the orchestration of said decisions by determining what tools and actions are needed to fulfil the prompt. This includes prompt construction and management, tool handling, execution sequencing, error management, and execution calling. Here, execution calling can refer to passing prompts to the Gemini API client, but it is also commonly used to interpret responses by the model, such as tool requests or textual responses. Within this structure, it uses the context provided by the session manager to construct an optimized prompt using that context.

A core feature is tool management and orchestration. This is done by registering available tools in the Tool Registry. It interprets tool use requests from the Gemini model and it is responsible for ensuring that the tool exists, that the request was valid, and that it can be invoked safely. Once having done so, the Agent Core/Orchestration subsystem is responsible for routing the appropriate calls with the provided arguments to the requested tools. Part of this process includes interpreting and setting the order in which the tools must run. Doing so ensures a proper sequence of actions that can be parsed through services in the integration layer. In addition, the security-driven architectural structure enacts certain critical principles, such as the need for explicit user approval before executing any tools that can modify files or execute shell commands. When the Agent Core/Orchestration subsystem receives the response, it can either dispatch execution to the appropriate tool or hand control back to the user.

Another vital service is the ability to correctly handle tool execution results or errors from the output of the integration layer by either returning results back to the model for further processing or parsing errors.

Again, as hinted at previously, the Agent Core/Orchestration subsystem is responsible for parsing prompts and executing calls to the specified tools based on the model's responses. In this layered architectural model, the actual execution is done in the integration layer. This means that there are various sub-subsystems within that layer that control different tooling components, as well as the Gemini LLM itself that is communicated with through the Gemini API Client in the Orchestration layer.. It is up to the Agent Core/Orchestration subsystem to send tool requests to the appropriate subsystems and construct prompts to be sent to the Gemini API client.

Other features of this subsystem include chat history compression, which will automatically compress the conversation history to be lossless before it is sent to the configured model in order to prevent exceeding token limits. The Agent Core/Orchestration is also partly responsible for model fallback, which allows the Agent Core/Orchestration to assess and decide to switch the user to the “flash model” for the current session if it senses that the user is being rate-limited by the default “pro” model. In another effort to minimize the strain put on the computational demand of the Agent Core/Orchestration subsystems, there have been pushes in recent updates toward sub-agent entities that operate within the main session and are instantiated dynamically by the Agent Core to handle specialized and complex tasks. While experimental as of writing this report (thus the lack of any further mention), it suggests the possibility of another sub-subsystem under the Agent Core that manages the invoking and tracking of these sub-agents. This again demonstrates a core trait of this layered architecture as well: evolvability.

2.2.4 Gemini API Client Subsystem

The Gemini API Client subsystem is primarily responsible for communicating with the Gemini API, thus acting as a bridge between the CLI and the API. By sending and receiving requests through an external service like the Gemini API, it provides an essential service. The Core Orchestration subsystem will send prompts to the Gemini API Client subsystem, which is responsible for wrapping the prompt into an appropriately formatted API request before submitting it to the API endpoint. The response is sent back through the API client and it is here that the response, which is either textual or a tool request, is conveyed back to the Agent Core/Orchestration subsystem. In addition to managing prompts and responses, the Gemini API client subsystem also handles the GEMINI_API_KEY and ensures safe, properly authenticated communication with the API. The Gemini API Client subsystem is also directly involved in the actual execution of the model fallback mechanism, as mentioned in the Agent Core subsystem description, in order to automatically switch models to create a seamless work experience for the end-user if the model has rate limitations.

2.2.5 Local Tooling Subsystem

The Tooling subsystem manages built-in tools that are implemented locally in the user's system. This allows direct interactions with the local environment through direct access to File System Tools, Shell Commands, Web Tools, and Execution Tools. Because we want to avoid unintended modifications, the Tooling subsystem is also responsible for enacting sandboxing mechanisms and other safety precautions whenever tools interact with the local system, such as with Execution Tools that require file modification or executes shell commands. Allowing direct interactions with local tools and resources enables integrated development, and this subsystem is instrumental in executing the processes that allow for AI capabilities beyond text generation. When the Agent Core/Orchestration subsystem is looking to send a tool request, it will route to the Tooling subsystem when it requires local tools that are already provided by the Gemini CLI and can be executed locally on the user's system.

2.2.6 External Tooling Subsystem

One of the advantages of Gemini CLI is that it can interact with external systems, tools, and data sources. The External Tooling subsystem uses an MCP server that introduces resources and tools to the Gemini CLI through the Model Context Protocol (MCP), which is a standardized method for connecting AI applications with external systems, labelled in the diagram above as two key components: Data Sources and Tools, and Other AI Applications. These "Data Sources and Tools" encompass external services like data and file systems (eg. PostgreSQL), development tools like Git, and productivity tools like Google Maps. Another grouping of external tooling that this subsystem integrates with are "Other AI Applications," which includes things like chat interfaces, AI driven platforms, and other LLMs like OpenAI or Claude.

Interplay with external components demonstrates how Gemini CLI can act as the client to other external systems' servers through the MCP. MCP servers act as a bridge between the Gemini model and the user's local environment, as well as providing access to services like APIs. The Agent Core/Orchestration subsystem might choose to route to the External Tooling Subsystem when it wishes to access externally hosted services or tools that cannot be accessed locally or are not already built into the Gemini CLI tool package, demonstrating the extensibility of Gemini CLI. Adopting the Model Context Protocol is particularly beneficial for developments by allowing integration between various remote ecosystems and the user, which can hasten development, improve AI capabilities across a multitude of systems, and improve overall end-user experience.

2.3 Interplay of Components

The CLI captures user input while the Session manager maintains authentication, tool permissions and conversation history so that each new request arrives at the Agent Core with the right context. They form an interface layer that handles terminal details and state tracking.

Once a request and its context reach the Agent Core/Orchestration subsystem, it becomes the central coordinator for all other parts. It reads data from the Session manager to construct prompts, decides whether a request should go to the Gemini API or to one or more tools and then sequences those calls so that results can be combined into a coherent answer. This process enforces safety and security constraints because the system pauses to ask the user for explicit approval before any tool that might modify files or run shell commands is allowed to proceed so decisions made at the core are always grounded in both context and user consent.

The Gemini API Client subsystem and the tooling subsystems are in the orchestration layer under the Agent Core/Orchestration subsystem. The API client turns the Agent Core's prompts into authenticated API requests, returns model outputs and can switch models when rate limits are hit. When tool use is needed, the Agent Core dispatches requests either to the local tooling subsystem (for on-machine actions) or to the external tooling subsystem (for MCP-based integrations). The Agent Core interprets their results or errors and then hands the combined outcome back up to the CLI creating a continuous loop in which each subsystem depends on the others but remains focused on its own clearly defined role.

Subsystems built on this architecture have separated uses. The CLI subsystem focuses on interaction and presentation, the Session manager concentrates on stateful aspects like authentication and context tracking and the Agent Core/Orchestration subsystem controls reasoning and decision-making in tool use. The Gemini API Client subsystem isolates external model communication. This reduces code duplication and fragmentation because each section is implemented once in its subsystem.

3 External Interfaces

The **Command-Line Interface (CLI)** acts as the primary external interface of the Gemini CLI application. Because the CLI adheres to standard UNIX stream conventions, it integrates naturally into makefiles, shell scripts, and CI/CD workflows (Gemini CLI, 2025). This enables tasks like code generation, analysis, and drafting to be performed automatically. Through a defined set of commands users can input prompts, context files, or configuration parameters to the CLI and receive outputs. Input is provided as command arguments or standard input, and returned in standard output which also allows for easy piping into other tools (Shiraishi et al., 2025).

The Gemini **backend API interface** exists between the Gemini API client subsystem in the execution layer and the remote LLM service. Communication occurs through authenticated HTTPS requests, which are managed by the Gemini API Client subsystem. This handles request formatting, streaming responses and error handling. Although the remote LLM may generate textual responses or propose tool invocations, it is strictly treated as an external source, it cannot directly access files, execute shell commands, or modify the local environment. To ensure controlled delegation, all proposed actions must return to the orchestration layer before any execution can occur.

Next is the **Local Operating System Interface**. This exists between the local tooling subsystem and the host operating system. Gemini CLI can read files, source code, and execute shell commands through this boundary. These capabilities enable use cases such as refactoring files, testing, and analyzing repositories. Before any environment-modifying action is performed, this interface is mediated by the orchestration layer and explicit user approval. The operating system, file system, shell, and version control environment are therefore considered external systems accessed only through controlled tooling abstractions.

The final external interface is the **External Services and MCP Integration Interface**, which exists between the external tooling subsystem and remotely hosted services such as search providers and API's. The Model Context Protocol (MCP) achieves integration through standardized protocols, which allows the Gemini CLI to connect to external data sources and service ecosystems. These services, accessed only after the orchestration logic validates and routes requests, remain outside the system boundary. Isolating these integrations in the execution layer supports the extensibility and modifiability, allowing new external services to be added without interface/orchestration restructuring.

3.1 Division of Responsibilities Among Developers

The responsibility division closely aligns with the layered structure of Gemini CLI and mirrors its four primary external interfaces. CLI interface developers manage the interface layer. They focus on command parsing, configuration, session display, and I/O behaviour. Due to this interface integrating with shells and CI/CD workflows, usability, stability, and backward compatibility become the main priorities. The general purpose of these developers is to ensure the CLI remains predictable even during backend evolution. Developers maintaining the Gemini backend API interface manage the API client within the execution layer. Their responsibilities include request formatting, authentication, and handling backend rate limits or API changes. Since the language model is an external service, reliability is critical. Tooling developers who implement file access, shell execution, and sandboxing mechanisms manage the local operating system and file system interface. They enforce approval gating and protect the user's environment from unintended side effects. Finally, developers working on the MCP integration

interface maintain adapters for remote APIs and third-party services. This work permits extensibility through new integrations without modifying orchestration or presentation logic.

4 Use Cases

4.1 Use Case 1: Refactor a File

Use Case Summary

A developer asks Gemini CLI to refactor a specific file. The CLI UI loads session context from the Session Manager and forwards the request to the Agent Core/Orchestrator. The Agent Core sends the prompt and available tools to the Gemini LLM via the Gemini API Client. The Gemini LLM requests that the file be read, and the Agent Core dispatches a read through Local Tooling to File System Tools. Using the returned file contents, the Gemini LLM produces a proposed patch (diff), and the CLI UI displays the diff to the user.

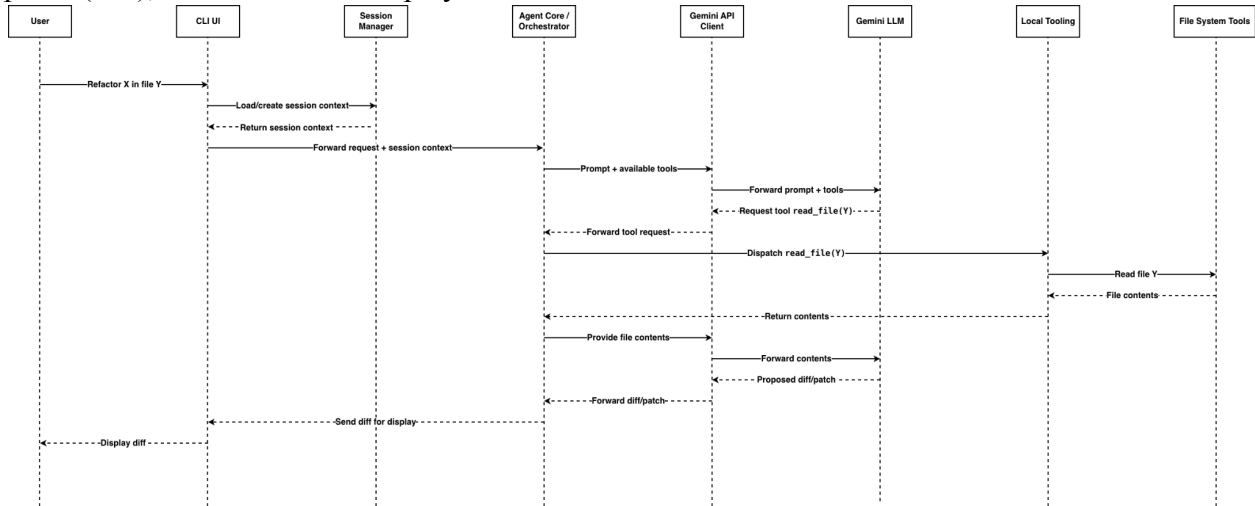


Figure 2: Sequence Diagram (Refactor X in File Y)

Link: [Sequence Diagram for Use Case #1](#)

Legend:

- **Solid message arrow:** request/command being issued (forward request, request tool call, dispatch read/write, show diff for approval)
- **Dashed message arrow:** returned data/result (file contents, tool results, write confirmation, final summary)
- **Vertical dashed lines (lifelines):** each participant's timeline; time flows top to bottom

Explanation: The CLI UI handles user interaction and display. The Session Manager provides persistent context. The Agent Core/Orchestrator controls the workflow and routes tool execution. The Gemini LLM proposes actions and generates the patch, but it does not access files directly. All file access occurs through Local Tooling using File System Tools.

4.2 Use Case 2: Run Tests Safely and Summarize Failures (Approval + Sandboxed Shell)

Use Case Summary

A developer asks Gemini CLI to run the test suite. The CLI UI pulls the session context from the Session Manager and sends the request to the Agent Core/Orchestrator. The Agent Core sends the prompt and available tools to the Gemini LLM through the Gemini API Client, and the Gemini LLM proposes a test command like run tests. The CLI UI shows that command for approval, and only after a yes does the Agent Core dispatch execution through Local Tooling to Shell Commands. Local Tooling returns stdout stderr and an exit code, then the Agent Core passes those results back through the Gemini API Client so the Gemini LLM can produce a short diagnosis and next steps. If the user denies approval, nothing runs.

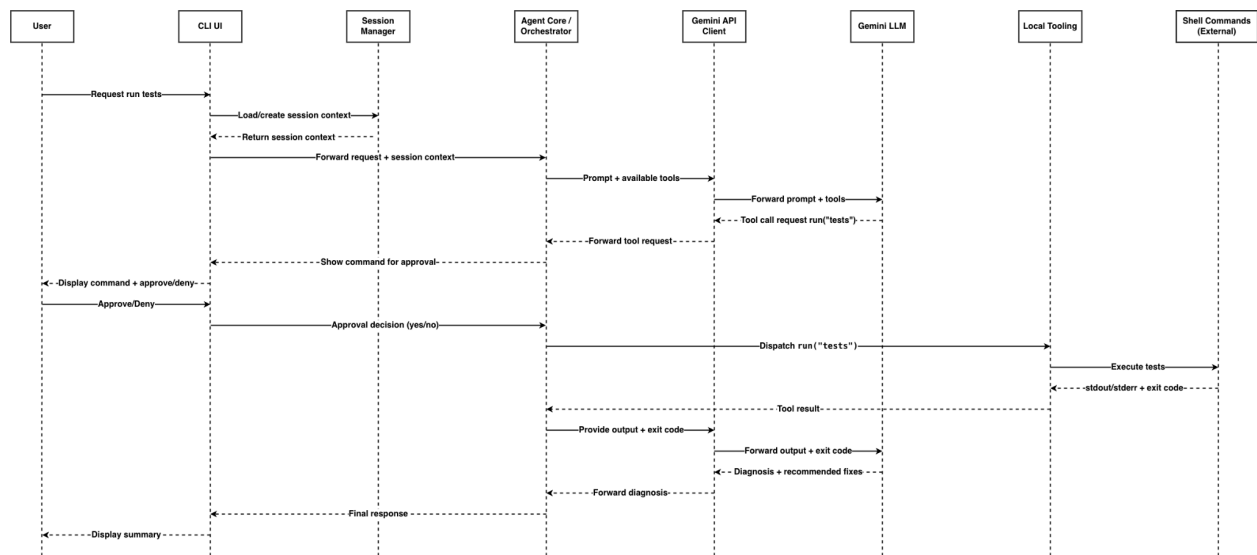


Figure 3: Sequence Diagram (Run Tests and Summarize Failures)

Link: [Sequence Diagram for Use Case #2](#)

Legend:

- **Solid message arrow:** request/command being issued (tool request, present command, dispatch execution)
- **Dashed message arrow:** returned data/result (stdout/stderr, exit code, structured result, diagnostic output)
- **Vertical dashed lines (lifelines):** each participant's timeline; time flows top to bottom

Explanation: The key point here is that the Gemini LLM can suggest what to run, but it cannot execute anything on its own. The approval step lives in the CLI UI, and the Agent Core/Orchestrator enforces that decision before Local Tooling is allowed to touch Shell Commands. The output from the test run is what the model uses to explain what failed and what to try next.

5 Conclusion

Gemini CLI's conceptual architecture is built around a simple control loop that matches how developers actually work. The CLI UI takes the request, pulls context from the Session Manager, and hands the task to the Agent Core Orchestrator. The Agent Core coordinates the workflow and communicates with the Gemini LLM through the Gemini API Client, while all file and shell actions are executed through Local Tooling and the appropriate external interfaces like File System Tools or Shell Commands. This split is the main strength of the design because it keeps responsibilities clear and makes side effects intentional. The model can propose actions, but it cannot directly touch the environment, and approvals are surfaced in the CLI UI before anything risky runs. A limitation of this architecture level view is that it leaves out reliability and performance behavior like streaming, retries, timeouts, and failure handling, which can matter in everyday use.

6 Lessons Learned

The group also gained a greater understanding of how modern AI-assisted technologies are constructed through analyzing the architecture of the Gemini CLI tool. One of the key things they recognized is the need to properly separate tasks involving user interaction, reasoning, and execution in AI-based systems. This is especially true for technologies that combine large language models and software. The experience of the application of this analysis also offers some insight into another challenge that was encountered. Specifically, while much of the publicly available documentation for the Gemini CLI tool was focused on usage and capability, there is less that is specific to the architecture itself. This meant that, for particular elements of the Gemini CLI architecture, such information had to be inferred based on the creation of a useful synthesis from the available information. From an architectural standpoint, the group also gained insight into how extensibility is a key concern when using LLM-based systems. For one, Gemini CLI's architecture demonstrated the significance of separating the agent core, tooling mechanisms, and external integrations to enable the system to adapt as the models and tools continue to grow and evolve. Additionally, there are trade-offs associated with using LLM-based architectures; they have varying performance and require external services. Another lesson learned was the value of early alignment on subsystem boundaries and abstraction level. At first, several members had variations in their understanding of how components should be organized and represented in the conceptual architecture. This caused some inconsistencies in terms and representation. By working together, the team was able to align their architectural model and use common names for sections. Looking back, it would have been beneficial to have achieved this alignment earlier.

7 Data Dictionary

- **Gemini CLI:** An AI-assisted command-line interface tool designed to support developers in performing software engineering tasks through natural language interaction and controlled tool execution.
- **Interface Layer:** The architectural layer responsible for user interaction and session management. It includes the CLI UI on the client side and the Session Manager on the server side.
- **CLI UI (Command-Line Interface User Interface):** The client-side component responsible for capturing user input, rendering responses, and presenting approval prompts.
- **Session Manager:** A server-side component that maintains session state, conversational history, authentication context, and tool permissions across user interactions.
- **Orchestration Layer:** The architectural layer responsible for reasoning, coordination, and workflow control. It includes the Agent Core/Orchestrator, Gemini API Client, Local Tooling, and External Tooling subsystems.
- **Agent Core/Orchestrator:** The central coordination component that interprets user intent, constructs prompts, sequences tool execution, enforces approval gating, and manages global control flow.
- **Gemini API Client:** The subsystem responsible for authenticated communication with the remote Gemini LLM service and handling model responses.
- **Integration Layer:** The architectural layer contains concrete execution targets and external systems. It isolates side-effecting operations from orchestration logic.
- **Local Tooling:** A subsystem responsible for executing approved actions within the local environment, such as file operations and shell commands.
- **External Tooling:** A subsystem responsible for connecting to externally hosted services and tools, including MCP-based integrations.
- **External Services:** Systems outside the Gemini CLI boundary, including the Gemini LLM backend, operating system services, file systems, shell environments, and remote APIs.
- **Large Language Model (LLM):** The remote AI reasoning engine used to generate responses and propose tool actions. It cannot directly execute side-effecting operations.
- **Layered Architecture:** An architectural style that structures a system into distinct layers with clear responsibilities and dependency direction.
- **Client–Server Architecture:** An architectural style in which client components request services from centralized server components through defined communication channels.
- **Agent-Oriented Architecture:** An architectural style in which an agent proposes or plans actions, while execution is delegated to controlled subsystems.

8 Naming Conventions

- **CLI (Command-Line Interface):** The primary interface of Gemini CLI, enabling command-based interaction, scripting, and integration with CI/CD workflows.
- **API (Application Programming Interface):** The interface used by Gemini CLI to communicate with the Gemini backend through authenticated requests.
- **CI/CD (Continuous Integration / Continuous Deployment):** Automated workflows in which Gemini CLI can be embedded to perform tasks such as code generation, analysis, and drafting.
- **STDIN / STDOUT (Standard Input / Standard Output):** Standard UNIX streams used by Gemini CLI to receive inputs and return outputs for piping into other tools.
- **LLM (Large Language Model):** Refers specifically to the remote Gemini backend responsible for language model inference and reasoning tasks.
- **NFR (Non-Functional Requirement):** Quality attributes such as performance, scalability, modifiability, maintainability, testability, usability, and reliability referenced in architectural justification.

9 References

Alateras, J. (2025, July 8). Unpacking the Gemini CLI: A high-level architectural overview. Medium. <https://medium.com/@jalateras/unpacking-the-gemini-cli-a-high-level-architectural-overview-99212f6780e7>

Gemini CLI. (2025). *Gemini CLI: Command-line interface for AI-powered workflows*. <https://geminicli.com/>

Fowler, M. (2018). *Continuous integration*. Martin Fowler. <https://martinfowler.com/articles/continuousIntegration.html>

Shiraishi, M., Cao, Y., & Shinagawa, T. (2025). *Effective command-line interface fuzzing with path-aware large language model orchestration*. arXiv. <https://arxiv.org/abs/2511.20555>

D. Garlan and M. Shaw, *An Introduction to Software Architecture*, Carnegie Mellon University, 1994.

L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed., Addison-Wesley, 2021.

Model Context Protocol. (n.d.). What is the Model Context Protocol (MCP)? <https://modelcontextprotocol.io/docs/getting-started/intro>

Queen's University, *CISC 322 – Architecture Styles Lecture Slides*, 2025.

Google, *Gemini CLI Documentation and Repository*, <https://github.com/google-gemini/gemini-cli>
ISO/IEC/IEEE 42010:2011, *Systems and software engineering — Architecture description*.

10 AI Report

1) AI Member Profile and Selection Process

The AI model we used was **OpenAI ChatGPT** running **GPT 5.2**. We chose ChatGPT because our biggest risk on A1 was inconsistency across artifacts, especially between the architecture diagram, subsystem descriptions, and the two sequence diagrams, and it helped us catch naming drift, missing handoffs, and unclear responsibility boundaries between the CLI UI, Session Manager, Agent Core Orchestrator, Gemini API Client, and Local Tooling. We also considered Google Gemini and Anthropic Claude. Gemini was thematically relevant, but we prioritized quick iteration and strict constraint following during diagram refinement, while Claude worked well for longer prose but was less aligned with our need for structure and consistency checking. Since A1 is conceptual rather than code-level, it depends on consistent vocabulary, clear boundaries, and a visible control loop that matches our architectural choices like client server, layered organization, and agent-oriented orchestration

2) Tasks Assigned to the AI Teammate

We delegated tasks that are repetitive and structure-heavy, where humans often miss small inconsistencies after multiple edits.

- Sequence diagram flow checks for both use cases. Reason: AI is good at spotting missing handoffs, unclear message direction, and steps that assign responsibility to the wrong subsystem.
- Naming consistency scans across artifacts. Reason: AI quickly flags mismatched subsystem names between diagrams and report sections.
- Readability tightening for diagram labels and surrounding text. Reason: AI can propose shorter labels and remove redundant messages while keeping meaning.
- Control loop visibility check. Reason: AI can verify the loop CLI UI to Orchestrator to Gemini LLM through API Client to Local Tooling to external systems and back is visible without extra explanation.

Boundaries: Humans made all final decisions. We did not rely on AI for internal facts about GeminiCLI beyond what we could justify with course sources or references.

3) Interaction Protocol and Prompting Strategy

One member acted as the primary driver for prompts to keep terminology stable. The rest of the team reviewed outputs and requested specific follow-up checks.

Workflow: We set a canonical vocabulary first, then reviewed one artifact at a time. We started with subsystem names, then Use Case 1, then Use Case 2, then the matching report paragraphs. We required the AI to stay at the architecture level and avoid invented subsystems.

Example prompt: Review this sequence diagram for correctness at the subsystem level. Only use these lifelines: User, CLI UI, Session Manager, Agent Core Orchestrator, Gemini API Client,

Gemini LLM, Local Tooling, External Systems. Identify missing handoffs, wrong responsibility assignments, and redundant messages. Keep labels short and stay conceptual.

Refinement: Early prompts were too open and produced extra components and implementation detail. We improved results by forcing a fixed lifeline list, banning invented subsystems, and requiring the same terms used in the report.

4) Validation and Quality Control Procedures

We did not accept suggestions because they looked right. Each change had to pass these checks:

- Mapping check: Every diagram lifeline must map to a subsystem described in the report or be labeled as an external system.
- Responsibility check: CLI UI handles interaction and approvals, Session Manager provides context, Agent Core Orchestrator controls workflow, Gemini API Client handles model communication, and Local Tooling is the only path to external systems like File System Tools or Shell Commands.
- Control flow check: Both use cases must show the same high-level loop and avoid implied side effects.
- Source discipline: Any statement that sounded like an internal implementation claim was removed or rewritten unless supported by a course source or a cited reference.
- Human review pass: At least one teammate who did not author the original diagram reviewed the diagram and nearby text for contradictions and naming drift

5) Quantitative Contribution to Final Deliverable

Our good-faith estimate is that ChatGPT contributed about 20 to 30 percent of the overall effort for A1, based on the parts where its suggestions directly changed diagram structure or wording compared to sections we wrote and verified entirely as a team. The highest contribution was sequence diagram flow checking and label tightening, where AI reduced iteration time but humans still redrew and finalized diagrams. Moderate contribution was naming consistency checks across report sections and diagrams. Lowest contribution was architectural analysis, subsystem definitions, citations, and final writing decisions, which were primarily human work.

6) Reflection on Human AI Team Dynamics

AI saved time by speeding up consistency checks and suggesting clearer wording quickly. It also created extra work because outputs required validation and occasional rewrites to avoid implementation detail. The AI helped brainstorming by offering alternative ways to express the same subsystem boundary, especially around who controls tool execution and how results return to the CLI. The main challenge was over-specific suggestions. We handled this by tightening prompts, keeping a fixed vocabulary, and rejecting details we could not justify. The key lesson for future deliverables is to treat AI like a fast reviewer, constrain it first, and rely on human verification for correctness and course alignment.